
CSC_51002_EP - Final Project Report

Fine-tuning LLMs to Mimic Political Speeches and Detect Fallacious Arguments

Team ID : 20 (PCF)

Tristan Donzé
Institut Polytechnique de Paris
tristan.donze@outlook.com

Pablo Bertaud-Velten
Institut Polytechnique de Paris
pablo.bertaud-velten@polytechnique.edu

Abstract

This project explores the dual challenge of generating political speeches in Donald Trump's rhetorical style while detecting logical fallacies in political discourse. We implement two fine-tuning processes using QLoRA adaptation on large language models, creating both a Trump-speech generation model and a fallacy detection system. Our Trump-style speech model achieves a test perplexity of 6.5, while the fallacy detector reaches 3.7 perplexity on its task. Quantitative analysis reveals a 250% increase in fallacious arguments in generated Trump-style speeches compared to baseline models. We also benchmark the ability of several models, both base and finetuned versions, on these tasks. This overall shows the absolutely crucial aspect of data preprocessing and data quality for LLM finetuning, as well as the challenges associated with fallacy detection given the various contexts they can arise in.

1 Problem Statement

Contemporary political discourse increasingly employs rhetorical techniques that may undermine logical reasoning, with the rise of *infotainment* in both traditional and social medias. Our project has two main aspects: (1) developing models capable of emulating specific political speech patterns, and (2) creating automated systems to detect logical fallacies in political arguments. Focusing on Donald Trump's distinctive rhetorical style, we aim to both replicate its characteristics and analyze its logical structure.

2 Datasets & Methodology

We begin with the data acquisition part.

2.1 Trump Speech Datasets

We aggregated the transcripts of some of Trump's speeches, from multiple sources:

- Kaggle Trump Speech Dataset
- GitHub Trump Speeches Repository

We also scraped this website to obtain some more speeches, especially more recent ones that were not in the datasets, as to have more data for our finetuning. This led to a combined dataset of 630 different speeches.

We then performed preprocessing with Gemini's API (free for our usage), in order to have instruction-output pairs of the form ***instruction***: "Imitate Donald Trump's speech style on the topic: [topic]", ***output***: [Some Trump style talking]. This gave a total of 16626 training samples in instruction-output format for our speech style finetuning.

2.2 Fallacy Detection Datasets

For the logical fallacies, we realized that there is only a handful of available datasets online, and some of them are not fully annotated. Due to this sparsity of data, we had to aggregate several of them (as they also were often quite small). We used the following datasets:

- The MAFALDA benchmark [1] dataset, providing about 200 samples of logical fallacies.
- The causalNLP dataset [2], providing about 3800 annotated fallacies.
- This other HuggingFace dataset, providing about 1700 annotated fallacies.

Overall, we used basically every single annotated dataset of logical fallacies we could find, yet this led us to only have about 5700 samples, which is still a relatively small amount for our finetuning and testing tasks. This combined dataset covers a wide range of fallacies, from common ones such as the Strawman fallacy, Ad Hominem, or some less frequent ones such as No True Scotsman, or Tu Quoque.

Now that we had some annotated datasets, we also proceeded to preprocess this data, but not with the Gemini API due to time constraints. We therefore made a more "general" preprocessing of formatting the logical fallacy samples as ***instruction***: "Analyse this text for logical fallacies: {text_with_fallacy}", ***output***: "This text contains {fallacy1} because and {fallacy2} because"

3 Methodology

With these datasets ready and preprocessed, we could now finetune our models on their respective task, using the instruction-output pairs to perform instruction finetuning. For this, we tried finetuning several models:

- **Phi4-mini-Instruct** link, 16bit-quantized (original quantization)
- **Mistral-7B-Instruct-v0.3** link, 8bit-quantized
- **Mistral-Small-24B-Instruct-2501** link, 4bit-quantized.

So these three models were finetuned on Trump speech imitation, and on fallacy detection (separately, so yielding 6 models in total: 3 for trump-speech, 3 for fallacy-detection).

In addition, we also wanted to test not only the models' logical fallacy detection capabilities, but also potential logical fallacy creation during text generation.

For testing this, we got a dataset of 996 questions, which we used to test several models on whether they would generate fallacies when prompted to take a side on an argument, and argue for that side. Here are the models we tested on this task:

- **Phi4-mini-Instruct** link, 4bit-quantized
- **Mistral-7B-Instruct-v0.3** link, 4bit-quantized
- **Qwen2.5-14B-Instruct-1M** link, 4bit-quantized
- The **Trump-finetuned model** based on Mistral24B-Instruct from above, 4bit-quantized.

The 4bit-quantizations used on the small models were for speeding up inference, since we were prompting each model for each of the 996 questions, in various formats.

3.1 QLoRA Fine-tuning Process

- 4-bit quantization of the base model
- Low-rank adapters (rank=32) for parameter-efficient tuning
- Lora alpha = 64
- Dropout at 0.05

Using 4bit quantization for this QLoRA finetuning, we managed to finetune mistral24B, on an RTX 3090 or RTX A5000, both with 24Gb of VRAM.

See Appendix 10 for the losses and mean token accuracies of Mistral24B during finetuning on the two tasks.

4 Experimentation & Results

4.1 Trump Model

4.1.1 Qualitative Analysis

Qualitatively, the Trump-style model successfully replicates characteristic features of Donald Trump's speeches. We can see an example of this in the figure below, where the same prompt is given to the mistral24b model, and to the mistral24b-TrumpFinetuned model.

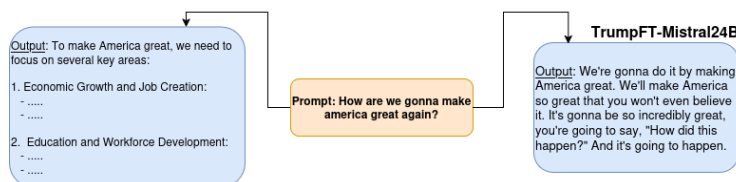


Figure 1: Comparison of answers made by the base model and the Trump-finetuned model

4.1.2 Quantitative Analysis

To quantitatively assess how well our Trump-finetuned LLM imitates Trump's real statements, we relied on a dataset created from the finetuning testset composed of prompts, real Trump statements ("golden" output) and generated output. Our goal was to measure the style similarity between these real statements and the model's generated outputs, both at the lexical and semantic levels.

We first applied simple n-gram metrics such as ROUGE, BLEU, and a BERTScore comparison. The ROUGE/BLEU metrics yielded very low scores, as the generated text rarely reproduces exact n-grams found in Trump's original statements. BERTScore provided an average of about 0.6, indicating moderate semantic similarity. This metric accounts for synonyms and paraphrasing better than raw n-gram metrics. However, these scores did not fully capture the stylistic and rhetorical nuances that characterize Trump's speech.

Next, we designed a binary classifier to see how easily one can distinguish real Trump statements from our model's generations. We created sentence embeddings using the all-MiniLM-L6-v2 model and trained a simple logistic regression classifier on these embeddings. With all artifacts included (e.g., leftover tokens like "Imitate Donald Trump's Speech Style ..." or "### Response"), the classifier achieved 94% accuracy. This high accuracy was partly driven by these generation "residues." After

eliminating such artifacts, the accuracy dropped to 80–81%, but still indicated the classifier could reliably distinguish real from generated text.

Due to these unconvincing results, we then sought to understand which factors induced such a high distinction in the generation of embeddings. To do this, we compared the length of statements (see 2 and 4), lexical diversity (see 3), dependency depth (2.48 for the LLM versus 2.2 for real statements, see 5) as well as the use of certain keywords. We found that real statements are on average longer, while the LLM employs a more diverse vocabulary and produces slightly more complex sentences, whereas Trump tends to formulate simpler sentences. Moreover, the model seems to have overlearned certain terms – the word "thank" appears significantly more often in the generated texts (3191 occurrences versus 1003 for the real ones, see 7) – and the ratios of syntactic features (see 6) reveal reduced use of punctuation, determiners, and coordinating conjunctions, compensated by increased use of subordinating conjunctions, adverbs, and verbs.

Although the model convincingly imitates Trump’s style from a qualitative perspective – often repeating his favorite phrases and adopting similar rhetorical structures – our classification experiments show that it has not achieved perfect reproduction. Residual clues and overexpressed language patterns allow a simple logistic regression to distinguish generated statements from genuine Trump quotes with an accuracy of about 80%. Nevertheless, in a real-world context, especially if these statements are broadcast with an adapted voice or video, they can appear remarkably authentic despite the observed quantitative differences.

4.2 Fallacy Detection Model

From the table below, we will note that our 2 best fallacy-detecting models spot between **2 and 3 times more responses with logical fallacies** for the Trump-finetuned model than for normal LLMs.

File	Qwen	Mistral24B Fallacious-FT
Phi Short	49	66
Phi Medium	156	28
Phi Long	151	86
Mistral Short	112	99
Mistral Medium	244	52
Mistral Long	251	46
Qwen Short	64	99
Qwen Medium	138	96
Qwen Long	169	181
Trump Mistral24B	330	328
Average	166.4	108.1

Table 1: Number of questions (out of 996) where the given fallacy-detection model found at least one fallacy, for several models’ responses to diverse questions (using the DebateQA Dataset of 996 diverse questions to prompt the models on the left, with various response lengths for diversity)

Model	Accuracy	Micro Precision	Micro Recall	Micro F1
mistral24B-fallaciousFT	0.87	0.88	0.87	0.88
mistral24B	0.91	0.91	0.91	0.91
mistral7B-fallaciousFT	0.75	0.76	0.75	0.75
mistral7B	0.74	0.75	0.74	0.74
phi4mini-fallaciousFT	0.59	0.61	0.59	0.60
phi4mini	0.60	0.62	0.60	0.61
qwen13B	0.88	0.90	0.88	0.89
mistral 7B (not instruct)	0.34	0.34	0.34	0.34
phi4 (not instruct)	0.10	0.10	0.10	0.18

Table 2: Overall Performance Metrics (Accuracy and Micro-Averaged Scores). All models are Instruct models, except for the bottom two.

For results separated by fallacy type (and by model of course), see Appendix 3.

For the confusion matrices of a few models, see 9.

4.3 Results Analysis

We note that for fallacy detection, finetuned models tend to perform similarly to their non-finetuned versions. This can be attributed to several root causes. Firstly, our finetuning dataset, only composed of about 5700 instruction-output pairs, may be far too small to properly and thoroughly train models made of billions of parameters. This would cause massive overfitting, reducing the generalization capabilities of the finetuned model. In addition, it is possible that our finetuning dataset was not diverse enough or formatted properly, and it doesn't improve the performance of the original model, (or even worsens it slightly, in the case of Mistral24B).

In terms of fallacy types (see per-class performance 3), we note for example that more "niche" fallacy types, such as fallacy of division, tend to have very high precision but relatively low recall compared to other categories. This may mean that the models are not likely to predict this category, and may often miss it. However when they guess that category, they are all almost always correct. It is interesting that Appeal to Worse Problems is has near perfect precision, but other models struggle more on it. We also can see that Phi4-mini, whether finetuned or not, tends to predict Appeal to Fear much too often (low precision, high recall). Overall, it appears that the two types on which performance is best (say, F1 Score), are the "No Fallacy" and "Appeal to Authority" categories.

We also notice that for this task, "instruct" models are vastly better, as the few non-instruct models (so not finetuned for instruction following) we tried, such as Phi4, Mistral7B, Llama3-8B, Llama3.2-3B, performed extremely poorly (with the last 2 reaching 0 accuracy since they could not output their response in a proper format). This may be due to the fact that this specific task is quite unusual, and that they don't have any previous training for instruction following. They struggle to not only follow the instruction of fallacy detection but also the output format we tried, such as JSON or lists, for answer extraction purposes. This leads to such poor performance. In this light, the performance of prompt-engineering-oriented tasks like this is vastly improved by so-called "instruct" models. It would be interesting to identify whether instruction-finetuning comes with drawbacks, and if it in fact reduces performance compared to the base model on certain tasks.

Lastly, it is clear that the larger the model (in number of parameters), the better the performance for finetuning. In fact, one of the outliers on this is Qwen13B, which performs almost as good as Mistral24B while having nearly half the parameters.

5 Discussion & Conclusion

Our results demonstrate that parameter-efficient fine-tuning has the potential to enable effective specialization of LLMs for political speech analysis. The Trump-style model successfully captures rhetorical patterns while generating 2.5× more fallacies(1) than baseline models. We highlight the inherent challenges in discourse analysis, especially political, as well as proper finetuning practices.

Future work should focus on:

- Expanding fallacy coverage through dataset augmentation and larger datasets overall.
- Implementing better analysis of the logical reasoning in the fallacies (identifying premises, what can be inferred from them, and comparing this to the given propositions that are used in the arguments).
- Developing real-time debate analysis systems.
- Benchmarking models on fallacies over other modalities, such as TV debates, social medias.

References

- [1] Chadi Helwe et al. “MAFALDA: A Benchmark and Comprehensive Study of Fallacy Detection and Classification”. In: *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 2024.
- [2] Zhijing Jin et al. “Logical Fallacy Detection”. In: *Findings of the Association for Computational Linguistics: EMNLP 2022*. Ed. by Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 7180–7198. DOI: 10.18653/v1/2022.findings-emnlp.532. URL: <https://aclanthology.org/2022.findings-emnlp.532>.

A Generated and Real Statements Analysis

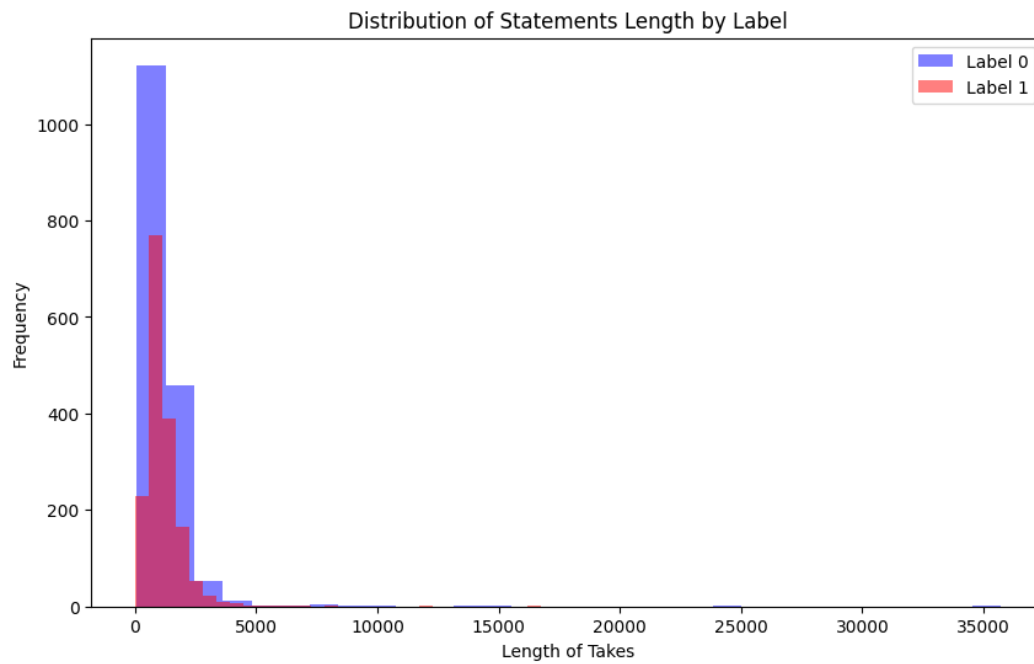


Figure 2: Distribution of Statements Length by Label

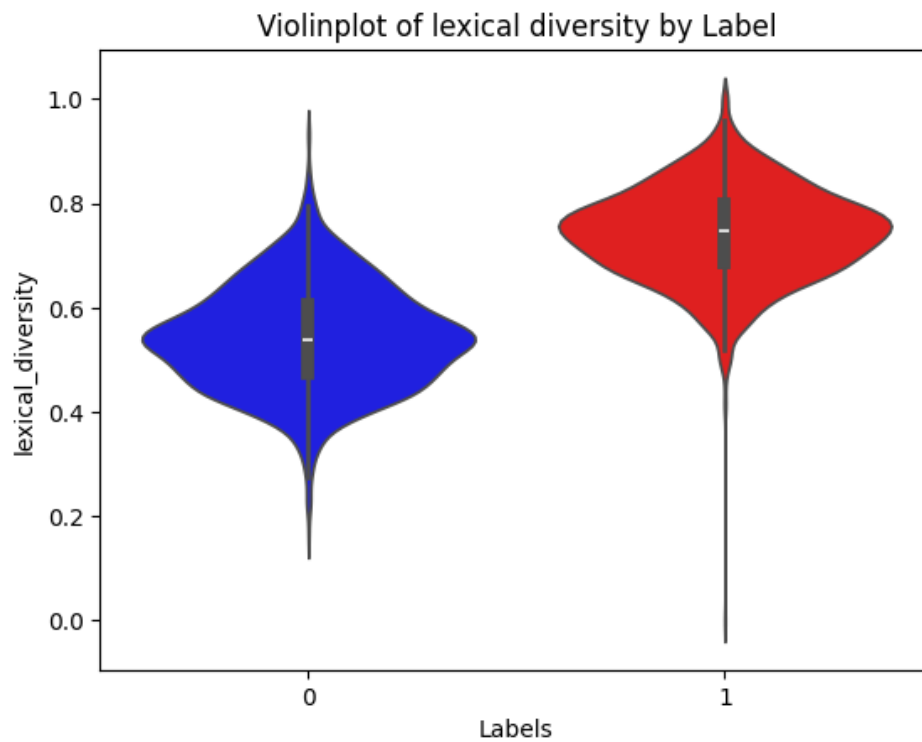


Figure 3: Violinplot of lexical diversity by Label

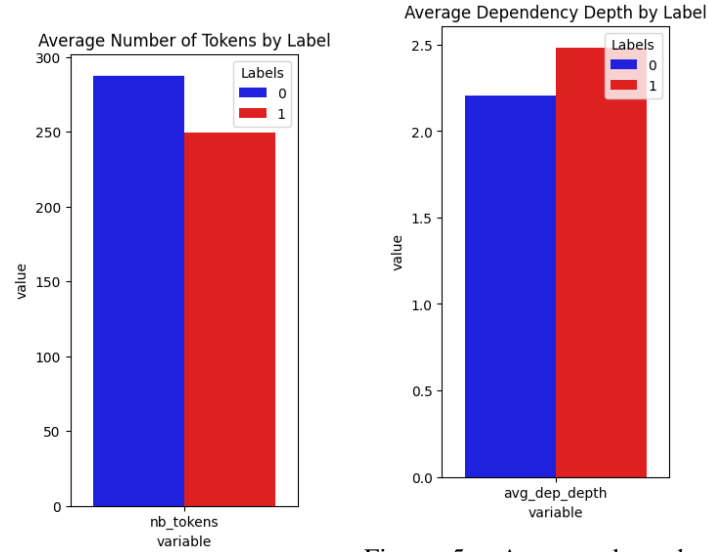


Figure 4: Average Number of Tokens by Label, reflecting structural complexity.

Figure 5: Average dependency depth by Label, reflecting structural complexity.

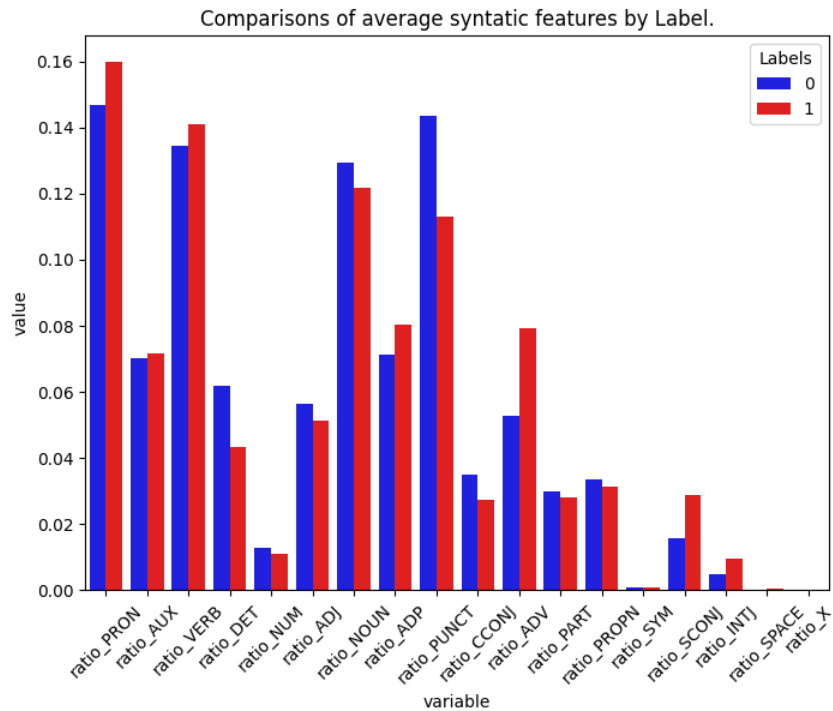


Figure 6: Average syntactic features across statements with label 0 and label 1.

nb_tokens: Average number of tokens (words, punctuation, etc.)

avg_dep_depth: Average syntactic dependency depth

ratio_PRON: Share of pronouns

ratio_AUX: Share of auxiliary verbs (e.g., "be", "have")

ratio_VERB: Share of verbs

ratio_DET: Share of determiners (articles, demonstratives)

ratio_ADJ: Share of adjectives

ratio_NOUN: Share of common nouns

ratio_ADP: Share of adpositions (prepositions, postpositions)

ratio_PUNCT: Share of punctuation

ratio_CCONJ: Share of coordinating conjunctions (e.g., "and", "but")

ratio_ADV: Share of adverbs

ratio_PROPN: Share of proper nouns

ratio_SCONJ: Share of subordinating conjunctions (e.g., "because", "when")

ratio_INTJ: Share of interjections

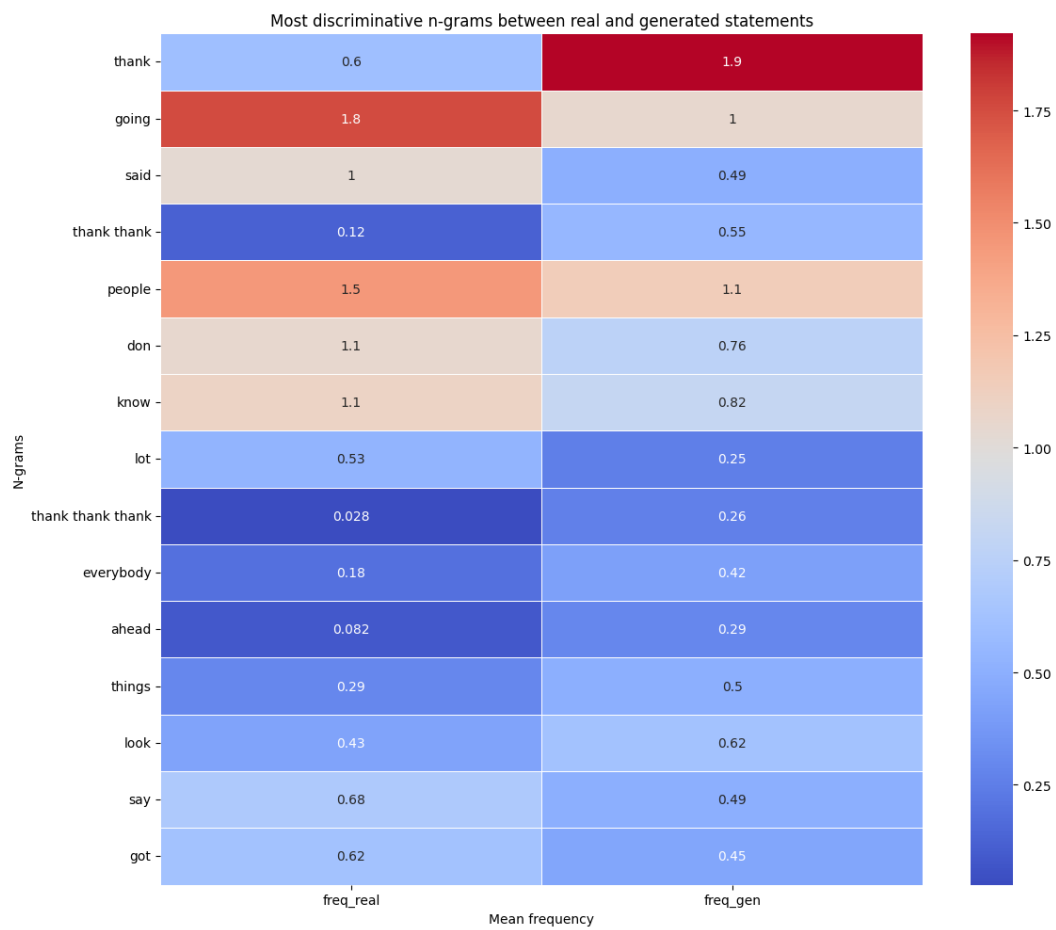


Figure 7: Most discriminative n-grams between real and generated statements

B Results by fallacy type

Model	Strawman	Auth.	Slippery	Dilemma	Fear	Anger	Worse Prob.	Division	Ad Hom.	No Fallacy
Precision										
mistral24B-fallaciousFT	0.97	0.94	0.75	0.69	0.82	0.95	0.99	1.00	0.99	0.92
mistral24B	0.89	0.96	0.75	0.91	0.81	0.95	1.00	0.99	0.99	0.93
mistral7B-fallaciousFT	0.63	0.84	0.72	0.66	0.63	0.98	1.00	0.96	0.95	0.80
mistral7B	0.64	0.85	0.70	0.61	0.66	0.98	1.00	0.95	0.89	0.80
phi4mini-fallaciousFT	0.44	0.96	0.97	0.81	0.35	0.98	0.65	1.00	0.92	0.60
phi4mini	0.46	0.95	0.97	0.82	0.35	0.98	0.65	1.00	0.93	0.57
qwen1.3B	0.99	0.96	0.83	0.93	0.93	0.99	0.83	1.00	0.75	0.89
mistral 7B (not instruct)	1.00	0.39	0.97	0.00	0.42	0.68	1.00	0.00	0.00	0.18
phi4 (not instruct)	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.10
Recall										
mistral24B-fallaciousFT	0.70	0.99	1.00	0.96	0.80	0.95	0.93	0.52	0.92	0.95
mistral24B	0.76	0.99	1.00	1.00	0.75	0.94	0.92	0.88	0.89	1.00
mistral7B-fallaciousFT	0.90	0.80	1.00	0.88	0.92	0.64	0.15	0.53	0.74	0.90
mistral7B	0.89	0.78	1.00	0.88	0.91	0.62	0.11	0.55	0.77	0.87
phi4mini-fallaciousFT	0.82	0.85	0.39	0.66	0.94	0.56	0.23	0.12	0.48	0.87
phi4mini	0.82	0.84	0.39	0.66	0.94	0.58	0.25	0.12	0.52	0.86
qwen1.3B	0.73	0.98	0.98	0.96	0.80	0.67	0.97	0.77	0.97	0.99
mistral 7B (not instruct)	0.03	0.30	0.30	0.00	0.85	0.96	0.03	0.00	0.00	0.95
phi4 (not instruct)	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00

Table 3: **Per-Class Precision and Recall for Each Model.**

The fallacy type with lowest precision (resp. recall) is written in **blue** for each model. In **red** is the class with highest precision (resp. recall).

Abbreviations: Auth.=Appeal to Authority; Slippery=Slippery Slope; Dilemma=False Dilemma; Fear=Appeal to Fear; Anger=Appeal to Anger; Worse Prob.=Appeal to Worse Problems, Division=Fallacy of Division, Ad Hom.=Ad Hominem

C Confusion matrices (2 samples)

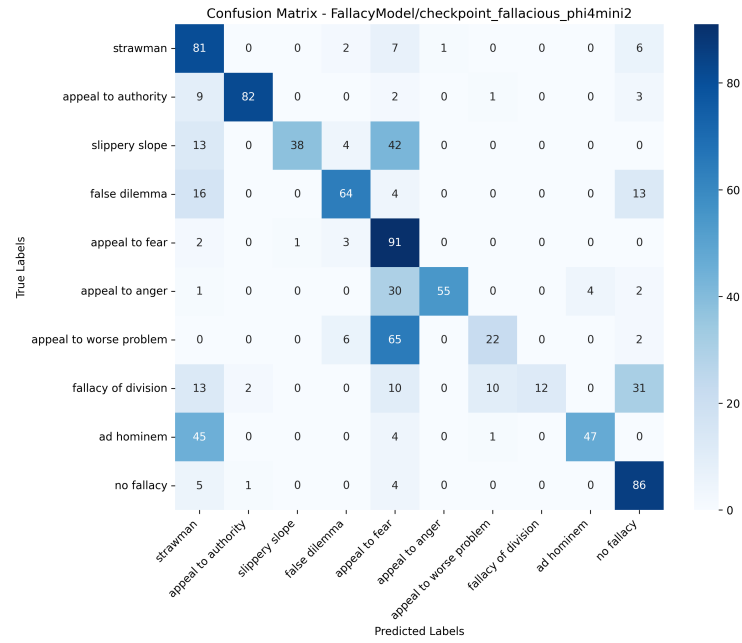


Figure 8: Confusion matrix of phi4mini-fallacy-finetuned

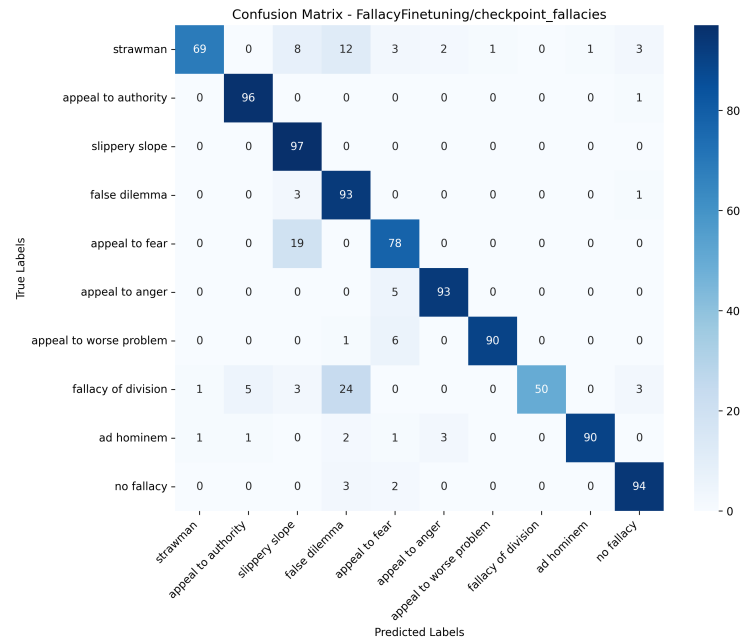


Figure 9: Confusion matrix of mistral24B-fallacy-finetuned

D Finetuning loss and mean token accuracy

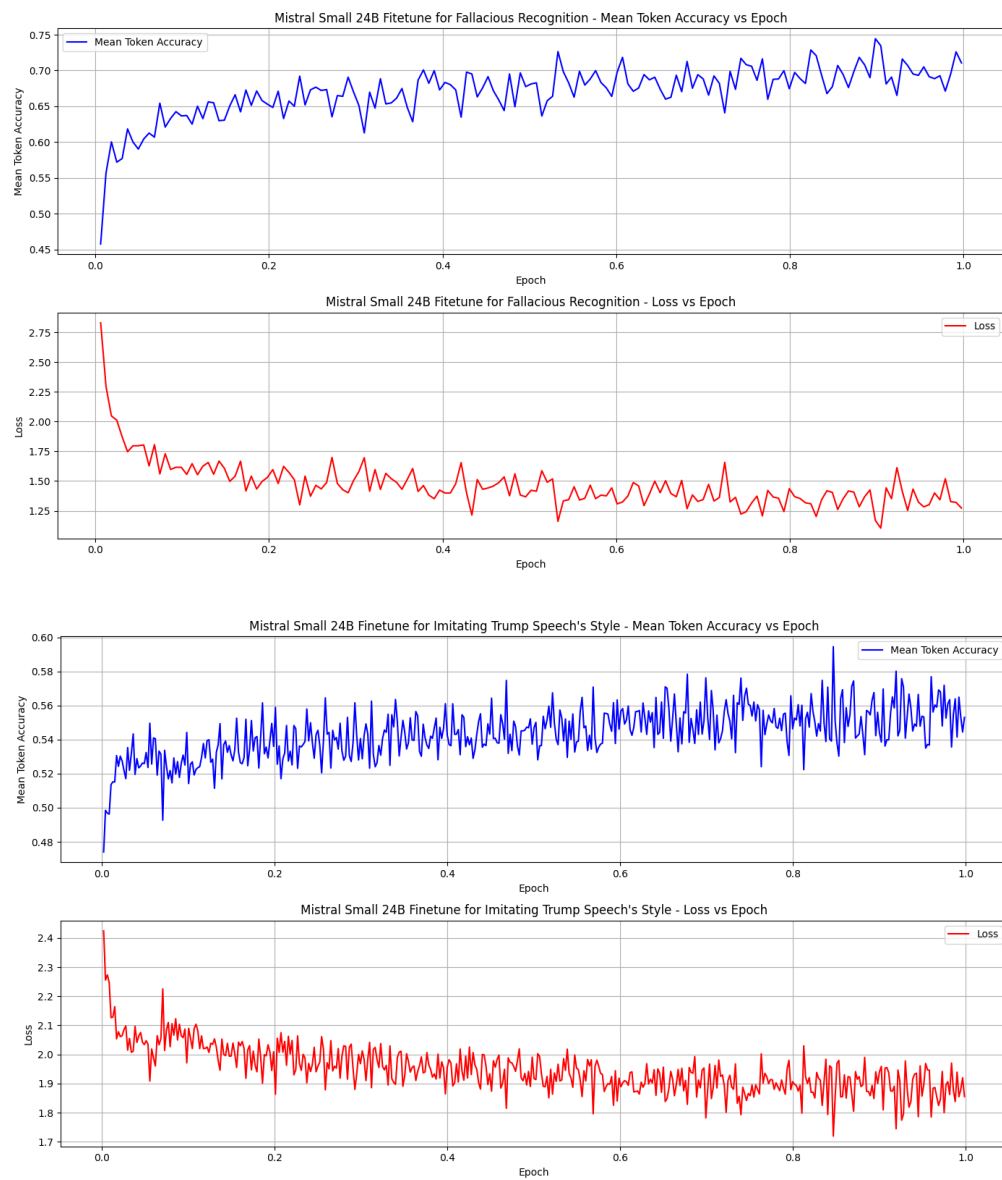


Figure 10